

Saibher API — Guía de conexión y autenticación

Versión del documento: 1.0 · **Módulo:** saibher_web_services · **Alcance:** API de Artículos (v1)

Esta guía te enseña, paso a paso, cómo conectarte a la API de Saibher: qué URL usar, cómo obtener tu credencial, cómo autenticarte en cada petición y cómo consumir los cuatro endpoints disponibles.

1. Introducción

La API de **Saibher Web Services** expone el tipo de contenido *Artículos* de saibher.com para que sistemas externos (apps, CRMs, automatizaciones, CI) puedan **leer, crear y actualizar** contenido de forma programática.

- Formato de intercambio: **JSON**.
- Autenticación: **API key propia** (esquema `Saibher`).
- Tres operaciones: consultar (`GET`), crear (`POST`) y actualizar (`PATCH`).

1.1 URL base

Entorno	URL base
Desarrollo (DDEV)	<code>https://saibher.ddev.site:8443</code>
Producción	sustituir por el dominio real (p. ej. <code>https://saibher.com</code>)

Todos los endpoints cuelgan de `/api/v1/articles` . Ejemplo de URL completa:

```
https://saibher.ddev.site:8443/api/v1/articles
```

1.2 Terminología

Término	Qué es
API key	Credencial global que identifica a un consumidor.
Key ID	Identificador público de la clave (24 caracteres hexadecimales).
Secret	Valor secreto (64 caracteres hexadecimales). Solo se muestra una vez al crearla o rotarla.
Credential	Lo que se envía en el header: <code><keyId>.<secret></code> .
Cuenta vinculada	Usuario de Drupal sobre el que se autentica la petición; hereda sus roles y permisos.

2. Conceptos de autenticación

La API usa autenticación mediante **API key** (no sesiones ni cookies). Cada petición debe incluir un header:

```
Authorization: Saibher {keyId}.{secret}
```

Ejemplo completo y real de una petición:

```
curl -H "Authorization: Saibher
14e67cbc9888a1789e580ed5.7a3e2b094e3067bdb43026048be4c1348ff814e9999ff60db0ac07d04228cb864b5ed867bcc4ad518d2d8d8a41a157
6c" \
https://saibher.ddev.site:8443/api/v1/articles
```

2.1 Cómo funciona por dentro

- 1. El módulo define el esquema `Saibher ( Authorization: Saibher ... )`.
- 2. Al recibir la petición, busca el `keyId` en el almacén de claves.
- 3. Compara el `secret` contra un **hash SHA-256 con sal** (el secret plano nunca se guarda).
- 4. Si es válida y no está revocada/expirada, la petición se autentica **como la cuenta de usuario vinculada** a esa clave.
- 5. La autorización final se decide con los **permisos de Drupal** de esa cuenta.

2.2 Resultados de autenticación

Situación en el header	Resultado
Sin header <code>Authorization</code>	401 Unauthorized (+ header <code>WWW-Authenticate: Saibher</code>)
Esquema distinto (<code>Basic</code> , <code>Bearer</code> ...)	401 Unauthorized
<code>keyId.secret</code> inválido, expirado o revocado	401 Unauthorized
Clave válida, pero la cuenta no tiene permisos	403 Forbidden

Nota de seguridad: 401 significa “no autenticado”; 403 significa “autenticado pero sin permiso”. Distinguirlos ayuda a diagnosticar problemas.

3. Obtener tus credenciales

Las claves se gestionan **sin desplegar código** mediante Drush o el panel de administración.

3.1 Opción A — Drush (recomendado para integraciones/CI)

En el entorno de desarrollo se usa `ddev drush` ; en servidores, `drush` directamente.

```
ddev drush saibher:api-key:add "Mi integración" --uid=774 --days=3650
```

Explicación de las opciones:

Opción	Descripción
<code>argumento label</code>	Nombre descriptivo del consumidor (aparece en la lista de claves).
<code>--uid</code>	Cuenta de usuario de Drupal a la que se vincula (o <code>--uid=<usuario></code>).
<code>--days</code>	Días de validez; <code>0</code> = sin expiración (por defecto).

Salida real (el secret se muestra **una sola vez**):

```
[OK] API key "Mi integración" created.

Key ID:      14e67cbc9888a1789e580ed5
Credential (Authorization value):

14e67cbc9888a1789e580ed5.7a3e2b094e3067bdb43026048be4c1348ff814e9999ff60db0ac07d04228cb864b5ed867bcc4ad518d2d8d8a41a1576c
```

⚠️ Copia la línea completa (`Credential`) en un gestor de contraseñas. **No se mostrará de nuevo**. Si la pierdes, rota la clave.

3.2 Opción B — Panel de administración

- 1. Entra en **Administración → Configuración → Servicios web → Saibher Web Services → API keys** (`/admin/config/services/saibher-web-services/keys`).
- 2. Pulsa **Añadir API key**.

- 3. Rellena: **Etiqueta**, la **cuenta de usuario vinculada** y los **días de validez**.
- 4. Guarda: el módulo te redirige a una página que muestra la credencial completa **una sola vez** (la borra de su almacén al mostrarla).

3.3 Rotar y revocar

Acción	Drush	Panel
Rotar (nuevo secret, invalida el anterior)	ddev drush saibher:api-key:rotate <keyId>	Botón “Rotar” en la edición de la clave
Revocar (inhabilitar de inmediato)	ddev drush saibher:api-key:revoke <keyId>	Desmarca “Clave activa”

Ambas acciones son **inmediatas**: las peticiones con una clave rotada/revocada reciben 401 desde ese mismo instante, sin cache ni despliegue.

4. Tu primera conexión

4.1 Listar artículos (GET)

```
curl -s -H "Authorization: Saibher $CRED" \
  "https://saibher.ddev.site:8443/api/v1/articles?limit=2"
```

Donde \$CRED es tu credencial completo (keyId.secret). Guardarlo como variable evita repetirlo:

```
export
CRED="14e67cbc9888a1789e580ed5.7a3e2b094e3067bdb43026048be4c1348ff814e9999ff60db0ac07d04228cb864b5ed867bcc4ad518d2d8d8a41a1576c"
```

Partes de la petición:

Parte	Explicación
curl -s	descarga silenciosa (sin barra de progreso)
-H "Authorization: Saibher ..."	header de autenticación
?limit=2	número de resultados por página (1-50)

4.2 Respuesta esperada (resumida)

```
{
  "data": [
    {
      "id": 212,
      "uuid": "3082ff4c-7f8c-4d71-a91f-1a14f94406fe",
      "type": "article",
      "title": "Cómo Descargar y Usar el Módulo Admin Toolbar en Drupal 10",
      "status": true,
      "created": 1717995953,
      "changed": 1717996220,
      "author": { "id": 1, "name": "saibher_developer" },
      "url": "https://saibher.ddev.site:8443/blog/drupal/...",
      "alias": "/blog/drupal/...",
      "category": [ { "id": 5, "name": "Drupal" } ],
      "tags": [ { "id": 10, "name": "Drupal" } ],
      "image": { "fid": 230, "url": "https://.../admin_toolbar.jpg", "alt": "..." },
      "summary": "Drupal es uno de los sistemas de gestión de contenidos..."
    }
  ],
  "meta": { "total": 41, "limit": 2, "offset": 0, "sort": "created", "dir": "desc", "language": "es" }
}
```

Si el header es incorrecto obtendrás en su lugar:

```
{
  "error": {
    "status": 401,
    "code": "unauthorized",
    "message": "Se requiere una API key válida. Envía el header \"Authorization: Saibher {clave}\"."
  }
}
```

5. Autenticación en profundidad

5.1 Header exacto

- Nombre: `Authorization`
- Valor: `Saibher` (con espacio) seguido del credential completo.

Válido	Inválido
<code>Saibher 14e67...ed5.7a3e...c6c</code>	<code>14e67...ed5.7a3e...c6c</code> (sin esquema)
<code>Saibher <keyId>.<secret></code>	<code>Bearer 14e67...ed5</code>
(case-insensitive del esquema)	<code>saibher 14e67...ed5</code> (el valor del secret sí es sensible a mayúsculas)

5.2 Respuestas de autenticación

- **401** se acompaña del header `WWW-Authenticate: Saibher` para que el cliente sepa qué esquema usar.
- **401** no revela ningún detalle interno (no se devuelve *por qué* falló en detalle).
- **403** indica permiso insuficiente de la **cuenta vinculada** (ver sección 6).

5.3 Registros (watchdog)

Cada evento queda registrado en el watchdog de Drupal:

- `Article @nid created via API by user @uid.` — creación.
- `Article @nid updated via API by user @uid.` — actualización.

- Intento con credencial inválida — error `401` de autenticación.

Útil para auditar **quién** (qué usuario) creó o modificó contenido.

6. Roles, permisos y cuentas de servicio

La cuenta vinculada a la API key necesita permisos en dos niveles:

A) Permisos del módulo (definidos en `saibher_web_services.permissions.yml`):

Permiso	Otorga
<code>access saibher web services</code>	Leer (GET).
<code>access saibher web services write</code>	Crear y actualizar (POST/PATCH).
<code>administer saibher web services</code>	Configuración y gestión de claves (solo administración).

B) Permisos de contenido de Drupal: las operaciones de escritura además exigen los permisos estándar de nodo de la cuenta (p. ej. `create article content`, `edit any article content`).

Ejemplo: en el entorno de desarrollo existen dos roles de referencia:

- `saibher_api_consumer` → lectura + escritura (usuario `api_svc`).
- `saibher_api_reader` → solo lectura (usuario `api_reader`).

Recomendaciones con cuentas de servicio

1. Crea **una cuenta por integración** (p. ej. `svc_facturacion`), no uses cuentas personales.
2. Asígnale el rol **mínimo necesario**: solo lectura para consumidores que no escriben.
3. Revisa la cuenta de servicio periódicamente como a un editor.
4. Usa claves con expiración para integraciones temporales (`--days`).

7. Referencia de endpoints

Método	Ruta	Permiso	Respuesta	Uso
GET	<code>/api/v1/articles</code>	lectura	200	Listar (paginado y filtrable)
GET	<code>/api/v1/articles/{article_key}</code>	lectura	200	Obtener uno por nid o UUID
POST	<code>/api/v1/articles</code>	escritura	201	Crear
PATCH	<code>/api/v1/articles/{article_key}</code>	escritura	200	Actualizar (parcial)

7.1 GET — Listar

```
GET /api/v1/articles?limit=10&offset=0&title=...&status=1&sort=created&dir=desc
```

Parámetro	Valores	Descripción
<code>limit</code>	1-50 (por defecto 10)	Tamaño de página
<code>offset</code>	≥ 0	Desplazamiento (paginación)
<code>title</code>	texto	Filtro por subtítulo (contiene, case-insensitive)
<code>status</code>	0 / 1	Filtra publicados/borradores (anula el ajuste del sitio)
<code>sort</code>	<code>nid</code> , <code>title</code> , <code>created</code> , <code>changed</code> , <code>status</code>	Campo de orden
<code>dir</code>	<code>asc</code> / <code>desc</code>	Dirección del orden

Respuesta: cuerpo con `data` (artículos `sin body`, con `summary` recortado a 255 caracteres) y `meta` con `total`, paginación y orden.

7.2 GET — Obtener uno

```
GET /api/v1/articles/212
GET /api/v1/articles/3082ff4c-7f8c-4d71-a91f-1a14f94406fe
```

Misma estructura que un elemento de la lista, pero incluye el campo `body` :

```
"body": { "value": "<p>...</p>", "summary": "...", "format": "basic_html" }
```

7.3 POST — Crear

```
POST /api/v1/articles
Content-Type: application/json
```

```
{
  "title": "Título del artículo",
  "body": {
    "value": "<p>Contenido HTML del artículo.</p>",
    "summary": "Resumen opcional",
    "format": "basic_html"
  },
  "status": true,
  "category": [ { "id": 5 } ],
  "tags": [ { "id": 10 }, { "name": "Tag nuevo (se auto-crea)" } ],
  "image": { "url": "https://ejemplo.com/foto.jpg", "alt": "Texto alternativo" }
}
```

- Respuesta `201 Created` con el artículo normalizado y header `Location` .
- `body` admite texto plano o el objeto `{value, summary, format}` .
- `format` debe ser uno de los permitidos en la configuración (`basic_html` , `restricted_html`). Cualquier otro devuelve `422` .
- `category` acepta `id` (términos existentes de `article_category`) o `name` (debe existir).
- `tags` acepta `id` o `name` ; los nombres desconocidos se **crean automáticamente**.

Campos escriturables (todo lo demás se ignora y se registra en el log):

Campo	Máquina	Tipo	Detalle
<code>title</code>	<code>title</code>	texto (≤255)	Obligatorio en POST
<code>body</code>	<code>body</code>	texto u objeto	<code>{value, summary, format}</code>
<code>status</code>	<code>status</code>	booleano	publicar (por defecto el ajuste del sitio)
<code>category</code>	<code>field_category</code>	lista	términos de <code>article_category</code>
<code>tags</code>	<code>field_tags</code>	lista	términos de <code>tags</code> (auto-crea)
<code>image</code>	<code>field_image</code>	objeto	<code>{url, alt}</code> ; la imagen se descarga y valida

 **Campos bloqueados:** `uid` , `author` , `created` , `changed` , UUID y revisiones **nunca** se aceptan del cliente; el autor y las fechas son siempre los de la cuenta vinculada a la clave.

7.4 PATCH — Actualizar (parcial)

```
PATCH /api/v1/articles/212
Content-Type: application/json
```

```
{
  "title": "Nuevo título",
  "tags": [ { "id": 10 } ]
}
```

- **Actualización parcial:** los campos ausentes no se modifican.
- Cada PATCH crea una **revisión nueva** del nodo (con log «Actualizado a través de la API de Saibher Web Services.»), lo que permite deshacer cambios.

8. Ejemplos de código

8.1 PHP (curl)

```
<?php
$base = 'https://saibher.ddev.site:8443/api/v1/articles';
$cred =
'14e67cbc9888a1789e580ed5.7a3e2b094e3067bdb43026048be4c1348ff814e9999ff60db0ac07d04228cb864b5ed867bcc4ad518d2d8d8a41a1576c';

function saibher_request(string $method, string $url, string $cred, ?array $body = null): array {
    $ch = curl_init($url);
    $headers = ['Authorization: Saibher ' . $cred, 'Accept: application/json'];
    $options = [
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_CUSTOMREQUEST => $method,
        CURLOPT_HTTPHEADER => $headers,
    ];
    if ($body !== null) {
        $options[CURLOPT_POSTFIELDS] = json_encode($body);
        $headers[] = 'Content-Type: application/json';
        $options[CURLOPT_HTTPHEADER] = $headers;
    }
    curl_setopt_array($ch, $options);
    $response = curl_exec($ch);
    $status = curl_getinfo($ch, CURLINFO_RESPONSE_CODE);
    curl_close($ch);
    return [$status, json_decode((string) $response, true)];
}

[$status, $list] = saibher_request('GET', $base . '?limit=5', $cred);
echo "HTTP $status, total: " . ($list['meta']['total'] ?? 'n/a') . "\n";

[$status, $created] = saibher_request('POST', $base, $cred, [
    'title' => 'Creado desde PHP',
    'body' => ['value' => '<p>Hola API</p>', 'format' => 'basic_html'],
]);
echo "HTTP $status, id: " . ($created['data']['id'] ?? 'n/a') . "\n";
```

8.2 Python (requests)

```
import requests

BASE = "https://saibher.ddev.site:8443/api/v1/articles"
CRED =
"14e67cbc9888a1789e580ed5.7a3e2b094e3067bdb43026048be4c1348ff814e9999ff60db0ac07d04228cb864b5ed867bcc4ad518d2d8d8a41a1576c"
HEADERS = {"Authorization": f"Saibher {CRED}", "Accept": "application/json"}

r = requests.get(BASE, params={"limit": 5}, headers=HEADERS)
r.raise_for_status()
data = r.json()
print("HTTP", r.status_code, "total:", data["meta"]["total"])

r = requests.post(
    BASE,
    headers={**HEADERS, "Content-Type": "application/json"},
    json={
        "title": "Creado desde Python",
        "body": {"value": "<p>Hola API</p>", "format": "basic_html"},
        "status": True,
        "tags": [{"id": 10}],
    },
)
print("HTTP", r.status_code, "id:", r.json()["data"]["id"])
```

8.3 JavaScript (fetch)

```
const base = "https://saibher.ddev.site:8443/api/v1/articles";
const cred =
"14e67cbc9888a1789e580ed5.7a3e2b094e3067bdb43026048be4c1348ff814e9999ff60db0ac07d04228cb864b5ed867bcc4ad518d2d8d8a41a1576c";

const opts = (method, body) => ({
    method,
    headers: {
        Authorization: `Saibher ${cred}`,
        Accept: "application/json",
        ...(body ? { "Content-Type": "application/json" } : {}),
    },
    ...(body ? { body: JSON.stringify(body) } : {}),
});

async function main() {
    const list = await fetch(`${base}?limit=5`, opts("GET")).then((r) => r.json());
    console.log("total:", list.meta.total);

    const created = await fetch(base, opts("POST", {
        title: "Creado desde JS",
        body: { value: "<p>Hola API</p>", format: "basic_html" },
    }));
    console.log("HTTP", created.status, "→", (await created.json()).data.id);
}

main();
```

9. Manejo de errores

Todas las respuestas de error siguen el mismo formato:

```
{
  "error": {
    "status": 401,
    "code": "unauthorized",
    "message": "Se requiere una API key válida. Envía el header \"Authorization: Saibher {clave}\",",
    "fields": { "title": "El campo \"title\" es obligatorio." }
  }
}
```

El campo `fields` solo aparece en **errores de validación** (422), con un detalle por campo.

HTTP	Código	Cuándo	Qué revisar
401	unauthorized	Credencial ausente, inválida, expirada o revocada	Header <code>Authorization: Saibher ...</code> , vigencia de la clave, <code>drush saibher:api-key:list</code>
403	forbidden	Credencial válida pero sin permisos	Roles/permisos de la cuenta vinculada
404	not_found	El <code>article_key</code> (nid/UUID) no existe	Identificador correcto
405	(Symfony)	Método no permitido (p. ej. <code>PUT</code>)	Usar uno de los 4 métodos soportados
415	request_error	<code>Content-Type</code> no es JSON	Enviar <code>Content-Type: application/json</code>
422	validation_failed	Payload con errores de validación	Campos devueltos en <code>fields</code>
500	internal_error	Error interno	Revisar watchdog; el mensaje nunca expone detalles

10. Seguridad y buenas prácticas

- **HTTPS siempre.** La credencial viaja en el header `Authorization`; nunca la pongas en la URL, en logs ni en repositorios (define como variable de entorno/secret del CI).
- **Rotación periódica.** Rota las claves en cada ciclo (p. ej. cada 90-180 días) con `drush saibher:api-key:rotate`.
- **Revocación inmediata.** Ante una fuga, revoca la clave al instante: `drush saibher:api-key:revoke <keyId>`.
- **Principio de mínimo privilegio.** Cuenta de servicio por integración y solo el rol necesario.
- **Rate limiting / WAF.** Protege `/api/v1/*` en el servidor proxy (p. ej. `limit_req` de Nginx). La columna *Last used* de `saibher:api-key:list` ayuda a detectar uso anómalo.
- **Auditoría.** Usa el watchdog para saber qué usuario creó/cambió qué artículo y cuándo.
- **Formatos de texto.** Solo `basic_html` y `restricted_html` están permitidos; valida que otros roles no puedan subir `full_html` por esta vía.
- **Imágenes remotas.** La descarga valida extensión, TLS y tamaño (`image_max_bytes`). El archivo queda registrado como uso para que no se purgue.

11. Postman

1. Importa `postman/saibher-web-services.postman_collection.json` (**Import**).
2. Define dos variables (nivel colección o entorno):
 - `baseUrl` → `https://saibher.ddev.site:8443`
 - `credential` → tu valor `keyId.secret`
3. La colección ya configura el header `Authorization: Saibher {{credential}}` en todos los requests.
4. Ejecuta: *Listar, Obtener, Crear, Actualizar* y los casos de error (401, 404).

12. Solución de problemas

Síntoma	Causa probable	Solución
401 siempre	Secret mal copiado / clave revocada/rotada	Vuelve a <code>saibher:api-key:add</code> (o rota) y copia el credential completo exacto
401 con esquema correcto	Expiración (<code>--days</code>) cumplida	Crea otra clave o sube el plazo
403 con clave válida	La cuenta vinculada no tiene lectura/escritura o permisos de nodo	Añade <code>access saibher web services (+ write)</code> y <code>create/edit article content</code>
422 <code>body.format</code>	Formato no permitido	Usa <code>basic_html</code> o <code>restricted_html</code> (o ajústalo en configuración)
415	Falta <code>Content-Type: application/json</code>	Añade el header en POST/PATCH
404 al actualizar	El <code>article_key</code> no existe	Usa un <code>nid</code> o <code>UUID</code> correcto
500	Error interno de la API	Revisa <code>ddev drush watchdog:show</code>
La credencial “se perdió”	El secret solo se muestra una vez	Usa <code>saibher:api-key:rotate</code> para generar uno nuevo

13. Apéndices

13.1 Referencia rápida de Drush

```
# Crear clave (la credencial se imprime una sola vez)
drush saibher:api-key:add "Etiqueta" --uid=774 --days=3650

# Ver claves y su estado / último uso
drush saibher:api-key:list

# Revocar una clave
drush saibher:api-key:revoke <keyId>

# Rotar una clave (nuevo secret)
drush saibher:api-key:rotate <keyId>
```

13.2 Glosario

Término	Definición
API key	Credencial de acceso a la API asociada a una cuenta de usuario.
Credential	Valor completo <code><keyId>.<secret></code> que viaja en el header.
nid	ID numérico interno del nodo/artículo en Drupal.
UUID	Identificador universal estable del artículo (aceptado como <code>article_key</code>).
Watchdog	Registro de eventos/dblog de Drupal.
Vocabulario	Conjunto de términos de taxonomía (<code>article_category</code> , <code>tags</code>).